

AWS Cloud Security Dashboard

Technical Documentation

Author: Mohamed Wasim Abdul

Version: 1.0

1. Project Objective

The objective of this project is to design and implement a serverless cloud security monitoring platform on AWS.

The platform continuously scans AWS resources for security misconfigurations, stores findings in DynamoDB, calculates a compliance score, generates alerts, and visualizes the overall security posture through a web dashboard.

The project demonstrates:

- AWS Cloud Architecture
 - Serverless Computing
 - Infrastructure as Code
 - Cloud Security Monitoring
 - Event-Driven Systems
 - Full Stack Development
-

2. High-Level Architecture

The solution consists of three major layers.

Frontend Layer

Responsible for visualizing security findings.

Components:

- React
- TypeScript
- Material UI
- Recharts

- Axios

Hosted on:

- Amazon S3 Static Website Hosting

Responsibilities:

- Display security score
 - Display findings
 - Show severity distribution
 - Show resource distribution
 - Show historical findings
 - Show security trends
-

API Layer

Provides access to findings stored in DynamoDB.

Components:

- API Gateway
- API Lambda

Endpoints:

GET /findings

GET /history

GET /summary

Responsibilities:

- Retrieve findings
 - Retrieve historical records
 - Calculate dashboard metrics
-

Security Engine Layer

Responsible for scanning AWS resources.

Components:

- EventBridge
- Scanner Lambda
- DynamoDB
- SNS

Responsibilities:

- Perform security scans
 - Store findings
 - Send alerts
 - Generate compliance score
-

3. Architecture Flow

Dashboard Request Flow

User



React Dashboard



Amazon S3



API Gateway



API Lambda



DynamoDB



Response Returned

Security Scan Flow

EventBridge Schedule



Scanner Lambda



IAM Scanner

S3 Scanner

Security Group Scanner



Generate Findings



Store Findings



Send SNS Alerts



Update Dashboard

4. AWS Services Used

AWS Lambda

Used for:

- Security scanning
- REST API backend

Reason:

Serverless architecture eliminates server management.

API Gateway

Used for:

- REST API exposure

Reason:

Provides secure HTTP access to backend services.

DynamoDB

Used for:

- Security findings storage

Reason:

Highly scalable NoSQL database with low operational overhead.

EventBridge

Used for:

- Scheduled execution

Reason:

Triggers automated security scans without manual intervention.

SNS

Used for:

- Security notifications

Reason:

Immediately notifies users when critical findings are detected.

S3

Used for:

- Static website hosting

Reason:

Cost-effective frontend hosting solution.

IAM

Used for:

- Security scanning
- Permissions management

Reason:

Allows inspection of users and policies.

5. Infrastructure as Code

Infrastructure is fully managed using Terraform.

Benefits:

- Reproducible deployments
- Version control
- Environment consistency
- Automated provisioning

Terraform provisions:

- Lambda Functions
 - API Gateway
 - DynamoDB
 - SNS
 - EventBridge
 - IAM Roles
 - S3 Website Hosting
-

6. Scanner Architecture

The Scanner Lambda serves as the central security engine.

Entry Point:

lambda_handler.py

Execution Flow:

run_all_scanners()

↓

IAM Scanner

↓

S3 Scanner

↓

Security Group Scanner

↓

Save Findings

↓

Calculate Score

↓

Send Alerts

7. IAM Scanner

File:

scanners/iam_scanner.py

Purpose:

Identify users with excessive privileges.

Current Check:

AdministratorAccess Assigned

Severity:

CRITICAL

Logic:

1. Retrieve all IAM users.
2. Retrieve attached policies.
3. Detect AdministratorAccess.
4. Generate finding.

Example:

User:

terraform-admin

Finding:

Administrator Access Assigned

8. S3 Scanner

File:

scanners/s3_scanner.py

Purpose:

Identify insecure S3 configurations.

Checks:

Bucket Encryption Disabled

Severity:

HIGH

Bucket Versioning Disabled

Severity:

MEDIUM

Logic:

1. List buckets.
 2. Verify encryption.
 3. Verify versioning.
 4. Generate findings.
-

9. Security Group Scanner

File:

scanners/security_group_scanner.py

Purpose:

Detect publicly exposed services.

Checks:

SSH Open To Internet

Port 22

Severity:

CRITICAL

RDP Open To Internet

Port 3389

Severity:

CRITICAL

MySQL Open To Internet

Port 3306

Severity:

CRITICAL

PostgreSQL Open To Internet

Port 5432

Severity:

CRITICAL

Logic:

1. Retrieve Security Groups.
 2. Inspect inbound rules.
 3. Detect 0.0.0.0/0 exposure.
 4. Generate findings.
-

10. Findings Storage Design

Database:

Amazon DynamoDB

Table:

security_findings

Attributes:

finding_id

resource_type

resource_id

severity

title

description

created_at

Purpose:

Persist findings across scans.

Benefits:

- Historical analysis
 - Trend generation
 - Compliance tracking
-

11. Compliance Score

The compliance score provides a simplified security health indicator.

Initial Score:

100

Penalties:

Critical = -15

High = -10

Medium = -5

Low = -2

Formula:

Compliance Score = 100 - Total Penalties

Minimum Score = 0

Example:

1 Critical

1 High

1 Medium

Score:

$100 - 15 - 10 - 5 = 70$

12. API Design

GET /findings

Returns:

Active findings only.

Purpose:

Dashboard view.

GET /history

Returns:

Historical findings.

Purpose:

Trend analytics and auditing.

GET /summary

Returns:

Security metrics.

Example:

Total Findings

Critical Count

High Count

Medium Count

Low Count

Compliance Score

13. Dashboard Features

Security Score Gauge

Purpose:

Provide executive-level security visibility.

Severity Distribution

Purpose:

Show risk distribution.

Resource Distribution

Purpose:

Show affected AWS services.

Active Findings

Purpose:

Display currently active security issues.

Historical Findings

Purpose:

Display previously recorded findings.

Trend Analytics

Purpose:

Show findings growth over time.

14. Security Benefits

The platform helps identify:

- Overprivileged IAM users
- Unencrypted S3 buckets
- Missing bucket versioning
- Publicly exposed Security Groups

Benefits:

- Continuous monitoring
 - Automated detection
 - Security visibility
 - Faster remediation
-

15. Deployment Process

Infrastructure Deployment

terraform init

terraform plan

terraform apply

Frontend Build

npm install

npm run build

Frontend Upload

aws s3 sync dist s3://bucket-name --delete

16. Infrastructure Destruction

To avoid AWS costs:

terraform destroy

Terraform removes:

- Lambda Functions
- API Gateway
- DynamoDB
- EventBridge
- SNS
- S3 Website
- IAM Resources

17. Lessons Learned

This project provided hands-on experience with:

- AWS Serverless Architecture
- Terraform Infrastructure as Code
- Security Monitoring Concepts
- DynamoDB Data Modeling
- Event-Driven Design
- REST API Development
- React Frontend Development
- Cloud Security Automation

18. Future Enhancements

Potential improvements:

- GuardDuty Integration
- CloudTrail Monitoring
- AWS Config Integration
- Multi-Account Monitoring
- CloudFront Distribution
- Automated Remediation
- PDF Security Reports
- Role-Based Access Control

Conclusion

The AWS Cloud Security Dashboard demonstrates a complete end-to-end serverless security monitoring solution built on AWS. The project combines cloud infrastructure, security automation, serverless computing, Infrastructure as Code, and frontend visualization into a single platform capable of continuously monitoring AWS resources and providing actionable security insights.